

Crypto HFT Platform

C++20 Trading Infrastructure for crypto.com

Smith
Investment
Fund

Table of Contents

1.	System Overview
2.	Core Engineering
3.	Testing
4.	Roadmap & Gaps

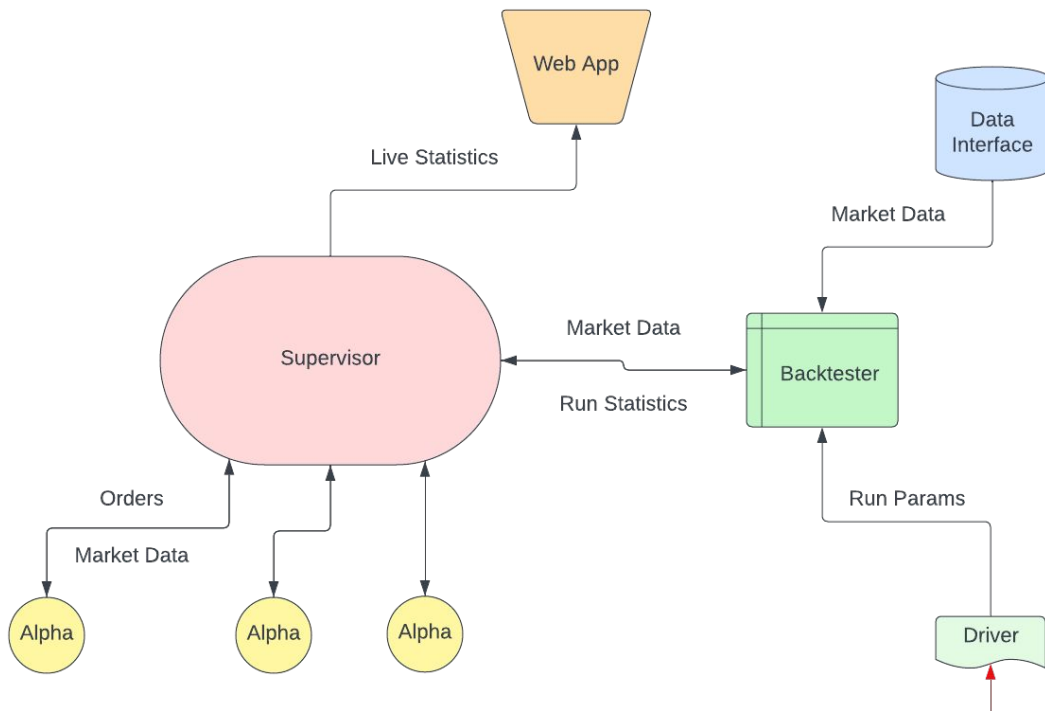
1. System Overview



What We Built

- An offline-testable C++20 MVP for crypto HFT: storage subsystem, Crypto.com market-data parsing, thread-safe order book, strategy runtime, simulated execution, risk manager, and a Unix-domain command server with a Python client.
- One strategy interface, two execution paths: the same strategy code runs live against the exchange via WebSocket order routing, or replays against a stored JSONL event log for backtesting.

High-Level Architecture



Architecture: Three Data Paths

- **Live trading path**

- Crypto.com market WebSocket → MarketDataClient → StrategyManager → strategy callbacks (on_quote / on_trade)
- RiskCheckedExecutionSink → LiveOrderExecutorSink → OrderExecutor → authenticated user WebSocket out to the exchange

- **Offline backtest path**

- JSONL event log → JsonlAsyncStorage replay → BacktestReplayDriver → same StrategyManager → SimulatedExecutionSink
- Same strategy code path; no exchange credentials required

- **Python control plane**

- hft_client.py → Unix domain socket (newline-delimited JSON) → UnixDomainCommandServer → subscribe / snapshot / ping callbacks

2. Core Engineering



Trade Format

- 64-byte cache-aligned struct, designed for lock-free queue throughput

```
struct alignas(64) Trade {
    uint64_t    client_order_id;    // monotonic ID
    uint32_t    strategy_id;        // source strategy
    char        ticker[16];         // e.g. "BTC_USDT"

    OrderSide   side;               // BUY or SELL
    OrderType   type;               // LIMIT, MARKET, STOP_LOSS
    TimeInForce time_in_force;      // GTC, IOC, FOK
    ExecType    exec_type;          // DEFAULT or POST_ONLY

    double      quantity;
    double      price;
    double      trigger_price;

    int64_t     timestamp_ns;       // ns precision
    uint8_t     priority;           // 0=highest, 255=lowest
};
```


Storage Subsystem

- IStorage interface: append, replay, flush, close, metrics. Append is non-blocking and thread-safe; offsets are monotonic for accepted writes.
 - InMemoryStorage: mutex-protected vector for fast tests and in-process simulation
 - JsonlAsyncStorage: bounded queue (default 65536), background flusher, batch size 256, flush interval 25 ms

JSONL line format

```
{  
  "schema_version": 1,  
  "offset": 42,  
  "ingest_ts_ns": 1730000000000,  
  "event_type": "quote_update",  
  "ticker": "BTC_USDT",  
  "payload": { ... }  
}
```

Strategy Runtime, Risk, and IPC

- StrategyBase: strategies inherit and override `on_quote` and `on_trade`. Each strategy declares the tickers it cares about and is dispatched by StrategyManager.
- RiskCheckedExecutionSink wraps any execution sink and enforces: positive quantity, max order quantity, max order notional, max absolute net position. Rejected orders never reach live or simulated execution.
- RiskManager tracks net quantity, average entry price, and realized PnL across fills.
- UnixDomainCommandServer accepts newline-delimited JSON commands: ping, subscribe, snapshot. The Python client (`hft_client.py`) sends commands and reads structured responses.
- The exchange layer signs every authenticated request with HMAC-SHA256, handles WebSocket heartbeats, and reconnects with exponential backoff.

3. Testing & Roadmap



Test Coverage

- Six test binaries built and run via ctest, all passing offline.
- test_storage (7 cases) covers in-memory round-trip, replay filters, monotonic offsets across producers, JSONL backpressure, flush and durability, close behavior, and malformed-line recovery.
- test_strategy_runtime, test_exchange_api, test_orderbook_risk, test_backtest, and test_ipc cover dispatch, message signing, order-book correctness, replay-driven backtests, and the Unix-domain command server.
- None of the test suites require live exchange credentials; backtests run against an empty or synthetic JSONL file.

Known Gaps and Next Steps (Translation Next Sem Trust)

- Gaps: Coinbase and other exchange connectors are not implemented; Crypto.com REST HTTP order management is not implemented; the UDS server class is tested but is not started by main yet.
- Gaps: simulated fills do not model queue priority, partial fills, exchange latency, slippage, or maker/taker classification realistically; live market data emits top-of-book quotes rather than full-depth book objects to the strategy layer.
- Next: a market-data-only collection mode that does not require private API keys, and a paper-trading mode that routes generated orders only to the simulated execution sink.
- Next: wire the UDS server into main, feed full book snapshots and deltas into the order book and expose them through the socket, add Docker and CI, and run sandbox soak tests before any production mode.